

ACTA DE ENTREGA DE SOFTWARE

Proyecto: Elevate — Plataforma de gestión gastronómica

Campo	Detalle
Proyecto	Elevate — Tienda web + Punto de venta (POS) + ERP financiero
Cliente	_____
Equipo de desarrollo	_____
Fecha de entrega	___ / ___ / 2026
Versión entregada	1.0 (rama <code>main</code> del repositorio)
Repositorio	<code>github.com/benjaminheredia1/elevateNext</code>

1. Objeto del acta

Este documento formaliza la entrega del sistema **Elevate** al cliente. Describe el alcance funcional entregado, el inventario técnico, las instrucciones de despliegue, operación y mantenimiento, los pendientes conocidos y las condiciones de aceptación.

Los anexos técnicos que acompañan esta acta son:

- **Anexo A — Guía de Usuario Final** (`docs/GUIA_USUARIO_FINAL.pdf`): manual de uso de todos los módulos, por rol.
- **Anexo B — Documentación de Arquitectura** (`docs/ARQUITECTURA_ELEVATE.pdf`): arquitectura técnica, estructura del código, seguridad y riesgos.
- **Anexo C — Documentos de diseño** (`docs/`): plan maestro, roadmap ejecutado y especificaciones por módulo.

2. Alcance funcional entregado

2.1 Tienda web (cliente final)

- Catálogo público multi-marca (`/menu/[marca]`) con imágenes, categorías y disponibilidad en tiempo real.
- Promociones y descuentos por **reglas horarias** aplicados automáticamente al precio mostrado.

- Flujo completo de pedido: armado del carrito, datos de contacto y entrega (delivery con coordenadas en mapa, o recojo en local).
- Seguimiento del estado del pedido.

2.2 Punto de venta / Caja (rol Cajero)

- **Turnos de caja:** apertura con monto inicial, cierre con arqueo (esperado vs. real por método de pago) y registro de diferencias.
- **Venta de mostrador (POS)** con total calculado en el servidor; métodos de pago efectivo, QR, tarjeta y **pago mixto** (efectivo + QR).
- **Numeración correlativa de pedidos por turno** (**#1..#n**), garantizada por índice único en base de datos.
- **Fiado (venta a crédito):** cuentas por cobrar ligadas a cliente registrado, con cobro total, parcial o selectivo, historial inmutable de pagos y sincronización automática del estado de pago de la venta.
- **Privilegios:** descuentos porcentuales elegidos por el cajero por venta, o aplicables posteriormente sobre una deuda.
- **Cortesías** (entrega sin cobro, con descuento de stock).
- Gestión de pedidos web: preparación, entrega en mostrador y despacho con repartidor (conciliación de efectivo de delivery por turno).
- Registro de gastos e ingresos extra del turno; libro de movimientos; historial de turnos cerrados con detalle.
- Alta y búsqueda de clientes desde caja.

2.3 Panel de administración / ERP (roles Dueño y Admin)

- **Dashboard ejecutivo** con KPIs, gráfico de pedidos por hora y pedidos recientes; notificaciones en vivo (nuevos pedidos y alertas de inventario).
- **Catálogo:** productos (con ciclo de vida: publicado, en revisión, baja lógica), categorías, recetas (ficha técnica por producto) y marcas.
- **Inventario:** insumos simples y mixtos (sub-preparaciones), unidades de medida con equivalencias, stock mínimo con alertas, costo promedio ponderado y movimientos (compras, mermas, conteos, producción).
- **Descuento automático de stock** por receta al preparar/entregar pedidos y en ventas de mostrador (idempotente: nunca descuenta dos veces).
- **Finanzas:** contabilidad (Estado de Resultados y Balance), flujo de caja por método y categoría, gastos fijos y operativos, ingresos extra, caja consolidada de todos los turnos.
- **Administración:** activos fijos con depreciación, cuentas por cobrar y por pagar, clientes (con deduplicación/fusión), privilegios, usuarios con roles, horarios de trabajadores.
- **Auditoría:** bitácora inmutable (solo-agregar) de toda acción sensible, con usuario, rol, fecha, monto, IP y navegador.
- **Rastreo de repartidores** en mapa (Leaflet) en tiempo real, mediante enlace por token sin necesidad de cuenta para el repartidor.

- **Analítica:** food cost e ingeniería de menú.

2.4 Transversal

- Autenticación con JWT y contraseñas cifradas (bcrypt); control de acceso por rol (DUEÑO, ADMIN, CAJERO, CLIENTE) verificado **en el servidor** en cada endpoint.
- Validación de toda entrada con esquemas Zod; límite de frecuencia de peticiones en operaciones sensibles.
- Precios y totales recalculados siempre en el servidor.
- Aplicación de escritorio para Windows (Electron) como envoltorio del POS.

3. Inventario técnico

Componente	Detalle
Framework	Next.js 16 (App Router) + React 19 + TypeScript
Base de datos	PostgreSQL (37 tablas), ORM Prisma 7
Estilos / UI	Tailwind CSS v4, PrimeReact, Framer Motion
Mapas / Gráficos	Leaflet / Recharts
Almacenamiento de imágenes	Vercel Blob
Backend HTTP	93 endpoints REST (<code>app/api/**</code>)
Pantallas	25 de administración, 13 de caja, tienda pública multi-marca
Tests	Vitest (unitarios, integración de endpoints y E2E de flujo) contra base de datos local desechable
Escritorio	Electron (<code>npm run electron:build</code> , Windows)

Estructura del código (detalle completo en el Anexo B): rutas HTTP delgadas en `app/api/**` → lógica de negocio en `lib/server/<dominio>/` → acceso a datos únicamente vía Prisma (`lib/prisma.ts`). Frontend en `app/`, `components/`, `hooks/` y `stores/`.

4. Entregables

1. **Código fuente completo** en el repositorio Git, con historial de cambios.
2. **Esquema de base de datos** (`prisma/schema.prisma`) y migraciones (`prisma/migrations/`), más datos semilla (`prisma/seed.ts`).

- 3. **Documentación:** esta acta y los anexos A, B y C (carpeta `docs/`).
- 4. **Scripts de operación:** respaldo (`scripts/db-backup.mjs`) y utilidades de mantenimiento (carpeta `scripts/`).
- 5. **Credenciales y secretos** (URL de base de datos, secreto JWT, tokens de servicios): se entregan **por canal seguro separado**, nunca dentro del repositorio.

5. Despliegue y operación

5.1 Infraestructura actual

Pieza	Dónde
Aplicación web	Vercel (build automático desde la rama <code>main</code>)
Base de datos	PostgreSQL autoalojado en VPS, gestionado con Dokploy
Imágenes de productos	Vercel Blob

5.2 Variables de entorno requeridas

Variable	Uso
<code>DATABASE_URL</code>	Conexión a PostgreSQL
<code>DIRECT_URL</code>	Conexión directa (migraciones)
<code>SECRET_JWT</code>	Firma de tokens de sesión
<code>SALT_ROUNDS</code>	Rondas de cifrado bcrypt
<code>BLOB_READ_WRITE_TOKEN</code>	Vercel Blob (imágenes)
<code>NEXT_PUBLIC_APP_URL</code>	URL pública de la aplicación
<code>RESEND_API_KEY</code> , <code>RESEND_FROM_EMAIL</code>	Envío de correos
<code>WHATSAPP_TOKEN</code> , <code>WHATSAPP_PHONE_ID</code>	Alertas WhatsApp (al activar proveedor real)

5.3 Comandos principales

```
npm install           # instalar dependencias
npm run dev           # desarrollo (usa la BD configurada en .env)
npm run build         # build de producción
npm start             # servir el build
npm test              # suite de tests (BD local desechable en Docker)
npm run electron:build # empaquetar app de escritorio para Windows
```

5.4 Entorno local de pruebas (sandbox)

```
docker compose up -d # PostgreSQL local (puerto 5435)
npm run db:local      # estructura + datos semilla
npm run dev:local     # desarrollo contra la BD local
```

El sandbox permite probar libremente sin afectar datos reales.

6. Mantenimiento

- **Cambios de esquema de BD** (flujo estándar; el historial de migraciones está verificado y al día con producción): editar `prisma/schema.prisma` → `npm run db:migrate` (genera y aplica la migración **en el sandbox local**) → revisar el SQL generado → aprobar el cambio → `npm run db:deploy` (`prisma migrate deploy`) contra producción como paso de release. **Nunca usar** `prisma migrate dev` ni `prisma db push` contra la base de producción, y el build de despliegue no debe tocar la base de datos.
 - **RespalDOS:** ejecutar `scripts/db-backup.mjs` de forma periódica y conservar los archivos fuera del repositorio.
 - **Calidad:** antes de publicar cambios, `npm run tsc --noEmit` y `npm test` deben pasar en verde. Toda corrección de bug debe incluir un test que lo reproduzca.
 - **Usuarios iniciales:** el seed (`prisma/seed.ts`) crea los usuarios base (administrador y cajero); las credenciales se entregan por canal seguro y deben cambiarse en el primer inicio de sesión.
-

7. Pendientes conocidos y recomendaciones

#	Punto	Estado / Recomendación
1	Alertas de inventario por WhatsApp	Implementadas en modo simulado; falta contratar y conectar un proveedor real (variables ya previstas).
2	Comando de build en <code>vercel.json</code>	Corregido en esta entrega: el build ya no modifica la base de datos (solo <code>prisma generate</code> + <code>next build</code>); las migraciones se aplican de forma controlada según la sección 6. Verificar que el panel de Vercel no tenga un build command propio que lo sobrescriba.
3	Sesión en el navegador	Ya implementado: la sesión viaja en cookie <code>httpOnly</code> + <code>secure</code> + <code>sameSite</code> (inaccesible para JavaScript, endurecida frente a XSS); el token no se guarda en <code>localStorage</code> .
4	Base de datos única	Resuelto por política de trabajo: el desarrollo se realiza contra el sandbox local (Docker) y producción solo se toca en el paso de release (<code>npm run db:deploy</code>).
5	Token de enlace del repartidor	Alargar el token y añadirle expiración.

Estos puntos no impiden la operación normal del sistema; se listan como plan de mejora priorizado.

8. Aceptación

Con la firma de la presente acta, el cliente declara haber recibido el código fuente, la documentación y los accesos descritos, y que el sistema entregado cumple el alcance detallado en la sección 2.

	Por el cliente	Por el equipo de desarrollo
Nombre	_____	_____
Cargo	_____	_____
Firma	_____	_____
Fecha	_____	_____